

PROJECT SYNOPSIS

(Specialization Project - C1 Evaluation)

Sentari Agent-OS

*(An Operating-System-Inspired Kernel for Scheduling, Isolation,
and Deadlock Detection in Multi-Agent AI Systems)*

GitHub Repository: <https://github.com/Vishalsym/Sentari-AgentOS>

Submitted by

Vishal Sharma

Roll No: 25225039

Under the guidance of

Dr. Ramesh Chandra Poonia

Dr. Shilpa Shrivastava



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE | DELHI NCR | PUNE

School of Sciences

2026-2027

PROJECT SYNOPSIS

1. Introduction

Modern agentic AI systems increasingly rely on multiple large-language-model (LLM) driven agents cooperating on a task. One agent researches, another drafts, another verifies. Frameworks such as LangGraph, CrewAI, and AutoGen make it easy to wire such agents together, but they operate purely at the application layer: agents are ordinary function calls with no notion of a managed process. There is no scheduler to divide API/compute resources fairly, no isolation between one agent's context and another's, no detection of circular waiting between agents, and no mechanism to forcibly stop an agent that has run away. Sentari Agent-OS proposes to close this gap by re-applying classical operating-system theory - process control blocks, preemptive scheduling, resource-allocation graphs, and a syscall-mediated access layer - to the problem of multi-agent orchestration.

2. Alignment with UN Sustainable Development Goals (SDGs)

The project aligns primarily with SDG 9 — Industry, Innovation and Infrastructure, since it contributes foundational software infrastructure that makes AI-agent deployments more resilient, auditable, and efficient — a prerequisite for trustworthy adoption of AI systems in industry. It carries a secondary alignment with SDG 12 — Responsible Consumption and Production: by enforcing token/API quotas and terminating runaway agents, Sentari Agent-OS directly reduces wasted compute and energy consumption caused by uncontrolled AI workloads.

SDG 9	Industry, Innovation and Infrastructure — building resilient, well-governed software infrastructure for AI-agent systems.
SDG 12	Responsible Consumption and Production — quota enforcement and preemption reduce wasteful compute/token usage.

3. Objectives

1. Design a process-control-block (PCB) model that represents each AI agent as a managed, schedulable entity with state, priority, and a resource quot
2. Implement a preemptive scheduler that fairly allocates LLM API calls / tokens across competing agents (round-robin and priority-based policies).
3. Build a syscall-mediation layer so agents never call tools or APIs directly — every action is validated, sandboxed, and logged by the kernel.
4. Implement a resource-allocation / wait-for graph with cycle detection to identify deadlocks between agents waiting on each other's output.
5. Implement preemption and forced termination for agents that exceed their quota, time out, or are found inside a deadlock cycle.
6. Provide isolated context (“virtual memory”) per agent with controlled, explicit sharing to a common knowledge base, preventing one agent's hallucinated state from silently corrupting another's.
7. Evaluate the system quantitatively — fairness under contention, deadlock-recovery time, and syscall-mediation overhead — against an unmanaged baseline.

4. Existing System and Its Limitations

Existing multi-agent frameworks (LangGraph, CrewAI, AutoGen, and similar) coordinate agents through direct function/method calls and shared Python objects. This works for demonstrations but exhibits the following limitations in practice:

- No resource governance: any agent can consume unlimited API calls/tokens, starving other agents or exhausting budget with no built-in throttling.
- No isolation: agents typically share mutable state directly; a hallucinated or corrupted value from one agent can silently propagate into another's context.
- No deadlock handling: if Agent A is waiting on Agent B's output and B is waiting on A's, the system simply hangs — there is no cycle detection or recovery path.
- No mediated tool access: agents typically call external tools/APIs directly, with no centralized validation, sandboxing, rate-limiting, or audit log.
- No preemption: a runaway agent (infinite loop, repeated retries) keeps running until it errors out or the process is killed externally by a human.

These are not hypothetical concerns — they are the most commonly reported operational failures in production multi-agent deployments today, and current frameworks address them, if at all, with ad-hoc try/except blocks rather than a systematic resource-management model.

5. Proposed System

Sentari Agent-OS proposes a thin kernel layer that every agent action must pass through, structured around four cooperating subsystems:

- Scheduler — maintains a PCB per agent and allocates LLM-API “turns” fairly using round-robin or priority-preemptive policies, directly analogous to CPU scheduling.
- Syscall Layer — the only path by which an agent may call a tool, read/write shared memory, or spawn another agent; every syscall is validated, logged, and rate-limited.
- Memory Manager — gives each agent an isolated logical context space with explicit, mediated access to a shared knowledge base, analogous to virtual memory paging.
- Deadlock Detector & Preemption Manager — maintains a resource-allocation graph across blocked agents, runs cycle detection to catch circular waits, and forcibly preempts or kills agents that deadlock or exceed their budget.

Unlike existing frameworks, Sentari Agent-OS does not replace LangGraph/CrewAI/AutoGen — it is designed as a middleware layer that can wrap their tool-calling and orchestration logic, making it adoptable without a full rewrite of an existing agent pipeline.

6. Specialization Concepts Applied

OS Concept	Applied As
Process Control Block (PCB)	Per-agent record: state, priority, token/API quota, owned resources
Preemptive / Priority Scheduling	Fair allocation of LLM API calls across competing agents
System Calls	Mediated tool/API access instead of direct agent-to-tool calls
Virtual Memory / Paging	Isolated agent context mapped to a shared knowledge base
Resource-Allocation Graph & Deadlock Detection	Cycle detection over agents waiting on each other's output
Process Termination & Signals	Forced preemption/kill of runaway or deadlocked agents

7. Expected Outcomes

- A working reference kernel demonstrating scheduling, syscall mediation, isolation, preemption, and deadlock detection over a multi-agent workload.
- Quantified fairness improvement (token/API distribution across agents) compared to an unmanaged baseline.
- Demonstrated detection and recovery from at least one induced circular-wait deadlock scenario.
- A reusable middleware pattern that can wrap existing agent frameworks with minimal integration effort.
- Complete technical documentation (SRS, design document) and a public GitHub repository for reproducibility.

8. Tools and Technologies (Proposed)

Layer	Technology
Core kernel	Python 3.11+ (asyncio for concurrency, dataclasses for PCB model)
LLM integration	Anthropic / OpenAI API (pluggable provider interface)
Agent frameworks wrapped	LangGraph / CrewAI (as external syscall targets)
Storage / logs	SQLite for PCB & syscall-log persistence
Dashboard (optional)	FastAPI + a lightweight web UI for live PCB / graph visualization
Testing	pytest, plus scripted fault-injection for deadlock/runaway scenarios

9. Repository

Project source, documentation, and evaluation scripts will be maintained at: <https://github.com/Vishalsym/Sentari-AgentOS>